

Requisitos do sistema novo — contrato de dados

2026-05-19. O nosso sistema lê os dados de cada lead, atribui um decil de qualidade (1–10) e dispara o evento de Conversão (CAPI) pro Meta com valor proporcional ao decil. Em 12/05 a captação trocou de tabela sem aviso e esse sinal morreu calado por dias — este é o contrato para isso não repetir. Tudo abaixo foi medido read-only no Railway em 2026-05-19; o que não foi confirmado está como "a confirmar".

1. Entrega dos dados

- **Um único payload por lead, com tudo junto:** identificadores (email, eventId) + fbp/fbc + hasComputer + userAgent + UTM (5 campos + url) + as respostas da pesquisa. Não queremos remontar o lead juntando tabelas por email — isso elimina a fragilidade de junção.
- **Ideal:** push por lead via Pub/Sub (tópico de nossa propriedade) — retry e dead-letter nativos, custo baixo, absorvido pela nossa própria API.
- **Fallback** (se Pub/Sub não for viável do lado de vocês): POST HTTP por lead, com retry+backoff e dead-letter **obrigatórios** — se o nosso endpoint piscar, o lead não pode sumir em silêncio.
- **eventId estável e único por lead** — o mesmo em qualquer reenvio do mesmo lead (não gerar um novo a cada tentativa/resubmissão). Vocês **não** precisam deduplicar antes de enviar; a deduplicação acontece do nosso lado e no Meta usando esse campo — mas só funciona se o eventId não mudar entre reenvios.
- Cadência do nosso lado: lote de 5 min (plano A); tempo real (plano B) só se o Meta confirmar ganho — ver §6.

2. Campos por lead — colunas limpas, não regredir

Hoje vêm da tabela `client` (~100% para quem responde pesquisa, medido n=1.588, 7d). O contrato é manter esses campos — entregues no payload único do §1 — sem regredir:

Campo	Tipo	Obrig.	Para quê
email	texto	sim	chave de junção entre as fontes (lowercase)
hasComputer	SIM/NAO	sim	feature crítica — sem ela o evento não dispara
eventId	texto	sim	dedup no Meta e no nosso registro
fbp	texto	sim p/ enviar	casamento da conversão no Meta
fbc	texto	recomendado	atribuição de clique (só existe se clicou em anúncio)
firstName, lastName, phone	texto	recomendado	dados pessoais do evento Meta
userAgent	texto	sim	dados do evento Meta + feature futura do modelo
ip4	texto	opcional	dados do evento Meta

3. Pesquisa — núcleo fixo

Uma coluna por pergunta (hoje em `lead_surveys`), vocabulário fechado. Estes valores **são** o contrato:

Coluna	Valores exatos
genero	Masculino · Feminino
idade	Menos de 18 anos · 18 - 24 anos · 25 - 34 anos · 35 - 44 anos · 45 - 54 anos · Mais de 55 anos
ocupacao	Sou CLT/Funcionário Público · Sou autonomo · Não trabalho e nem estudo · Sou apenas estudante · Sou aposentado
faixaSalarial	Não tenho renda · Entre R\$1.000 a R\$2.000 · Entre R\$2.001 a R\$3.000 · Entre R\$3.001 a R\$5.000 · Mais de R\$5.001 (reais ao mês)
cartaoCredito, estudouProgramacao, faculdade, investiuCurso	Sim · Não
atracaoProfissao	Poder trabalhar de qualquer lugar do mundo · Todas as alternativas · A possibilidade de ganhar altos salários · Trabalhar para outros países e ganhar em outra moeda · A ideia de nunca faltar emprego na área
interesseEvento	Fazer transição de carreira e conseguir meu primeiro emprego na área · Fazer um projeto na prática · Quero saber se é para mim · Fazer freelancer como programador · A aula com a recrutadora

- Campo do núcleo **não pode chegar vazio** (hoje ocorrem alguns ' ' — é defeito de formulário a corrigir).
- Perguntas **extras por lançamento** são permitidas, mas isoladas e fora do núcleo (o modelo só consome o núcleo).
- Mudar o núcleo (texto da pergunta, opções, renomear, "Sim/Não"→"S/N") só com mudança de contrato + replanejamento de modelo.

4. UTM — crua, nunca normalizada por vocês

- Mandar UTM **exatamente como veio do anúncio** (source, medium, campaign, content, term + url). Se vier agrupada, perdemos a medição de desvio treino↔produção.
- Origem desconhecida **não quebra**: vira "outros" do nosso lado — o gestor pode testar campanha/público novo à vontade.
- **Não mandar placeholder/literal de template não-renderizado** (ex.: [field id="utm_source"], utm_source, { . . . }) nem ID numérico cru como valor de UTM — se não houver valor, mandar vazio. Lixo de template infla a categoria "outros" e cega a medição de desvio.
- Informativo (decisão nossa, não exige ação de vocês): só disparam ao Meta as origens facebook-ads, instagram, ig, fb. A campanha LEAD | LQ é bloqueada **de propósito** para ser uma campanha de controle sem eventos de ML — mantém um fluxo de leads que **não** foram atraídos pelo modelo, evitando um feedback loop em que o modelo passa a treinar só com leads que ele mesmo trouxe; a decisão está descontinuada no momento, mas o ideal é manter sempre ~5% do orçamento numa campanha de controle sem eventos de ML.

5. Processo (o contrato de dados)

- Toda mudança no contrato (perguntas, campos, schema, link de captação, UTM) avisada a **todas as partes** (nós, dev, gestor de tráfego, ActiveCampaign/WhatsApp) **antes** de ir a produção.
- Schema com número de versão + registro de mudanças com data (lista de "o que mudou e quando").
- Ambiente de teste (staging) onde uma mudança de contrato é validada com a gente **antes** de ir a produção — para não estrear a mudança direto em produção e derrubar o sinal calado (foi o que aconteceu em 12/05). O teste tem que ser com **payloads reais passando pelo modelo e gerando resposta** (decil), não um teste superficial de schema.

6. A confirmar

- Frescor pro Meta: lote 5 min vs tempo real.
- Nome da tabela nova que será usada.
- Papel de LeadsClient (parou de receber dados em 18/05).

Anexo A — Payload completo: formato JSON e valores aceitos

Um objeto JSON por lead. Como enviar (que requisição, em qual endereço): **Anexo C**. Abaixo o **formato exato do JSON** que o nosso sistema/modelo espera, e depois a lista de **todos os valores aceitos** para cada campo de vocabulário fechado.

Formato do JSON (estrutura exata)

```
{
  "eventId": "b7e8c1a2-4f56-4a90-9c3d-1e2f3a4b5c6d",
  "submittedAt": "2026-05-22T18:21:09.524Z",
  "email": "fulano@exemplo.com",
  "firstName": "Fulano",
  "lastName": "de Tal",
  "phone": "+5567992956396",
  "hasComputer": "SIM",
  "fbp": "fb.2.1779154667671.108237608398356123",
  "fbc": "fb.2.1779154667652.IwZXh0bg...",
  "userAgent": "Mozilla/5.0 (Linux; Android 15; ...) ...",
  "ip4": "201.71.5.49",
  "survey": {
    "genero": "Masculino",
    "idade": "25 - 34 anos",
    "ocupacao": "Sou CLT/Funcionário Público",
    "faixaSalarial": "Entre R$2.001 a R$3.000 reais ao mês",
    "cartaoCredito": "Sim",
    "estudouProgramacao": "Não",
    "faculdade": "Sim",
    "investiuCurso": "Não",
    "atracaoProfissao": "Poder trabalhar de qualquer lugar do mundo",
    "interesseEvento": "Fazer um projeto na prática"
  },
  "utm": {
    "source": "facebook-ads",
    "medium": "Aberto",
    "campaign": "DEVLF | CAP | FRIO | FASE 01 | ADV | ML",
    "content": "DEV-AD0027-vid-captação-V1-DEV-REACT",
    "term": "fb",
    "url": "https://lp6.rodolfomori.com.br/meta-inscricao-lf-crt-v1/?utm_source=facebook-ads"
  }
}
```

Cada campo de vocabulário fechado (hasComputer, todos os de survey, utm.source, utm.medium, utm.term) aceita **apenas** um dos valores listados nas tabelas abaixo. Valor fora da lista vira outros no encoding e perde sinal.

A.1 Top-level — identidade, captura Meta, telemetria

Campo	Tipo	Obrig.	Formato / valor esperado
eventId	string (UUID)	sim	UUID v4 ou v7. Estável e único por lead — Anexo B.
submittedAt	string (ISO-8601 UTC)	sim	Ex.: "2026-05-22T18:21:09.524Z"
email	string	sim	trim + lowercase
firstName	string	recomendado	livre
lastName	string	recomendado	livre; "" aceitável

Campo	Tipo	Obrig.	Formato / valor esperado
phone	string	recomendado	dígitos; ideal E.164 ("5567992956396")
hasComputer	string	sim (crítico)	exatamente "SIM" ou "NAO"
fbp	string ou null	sim p/ Meta CAPI	"fb.<sub>.<ts>.<id>" ou null
fbc	string ou null	recomendado	"fb.<sub>.<ts>.<id>" ou null (nunca "")
userAgent	string	sim	navigator.userAgent cru
ip4	string	opcional	IP da requisição (backend)
survey	objeto	sim	ver A.2
utm	objeto	sim	ver A.3

A.2 survey — vocabulário FECHADO (10 features do modelo)

Mande **exatamente** estas strings. A normalização interna (unidecode + lower) achata acentos/capitalização, mas envie na forma da tabela pra evitar surpresa.

Coluna	Valores esperados (todos)
genero	Masculino, Feminino
idade	Menos de 18 anos, 18 - 24 anos, 25 - 34 anos, 35 - 44 anos, 45 - 54 anos, Mais de 55 anos
ocupacao	Sou CLT/Funcionário Público, Sou autonomo, Não trabalho e nem estudo, Sou apenas estudante, Sou aposentado
faixaSalarial	Não tenho renda, Entre R\$1.000 a R\$2.000 reais ao mês, Entre R\$2.001 a R\$3.000 reais ao mês, Entre R\$3.001 a R\$5.000 reais ao mês, Mais de R\$5.001 reais ao mês
cartaoCredito	Sim, Não
estudouProgramacao	Sim, Não
faculdade	Sim, Não
investiuCurso	Sim, Não
atracaoProfissao	Poder trabalhar de qualquer lugar do mundo, Todas as alternativas, A possibilidade de ganhar altos salários, Trabalhar para outros países e ganhar em outra moeda, A ideia de nunca faltar emprego na área
interesseEvento	Fazer transição de carreira e conseguir meu primeiro emprego na área, Fazer um projeto na prática, Quero saber se é para mim, Fazer freelancer como programador, A aula com a recrutadora

A.3 utm — cru do anúncio, NÃO normalizar do lado de vocês

Manter como veio do ?utm_*=... do anúncio. Onde o modelo tem vocabulário fechado, listei **todos** os valores que ele aprendeu (Champion jan30 + Challenger abr28). Qualquer valor fora dessas listas é mapeado pra outros pelo nosso encoding — funciona, mas o sinal vira genérico.

utm.source — valores reconhecidos pelo modelo:

Modelo	Valores aprendidos
Champion jan30	facebook-ads, google-ads, outros
Challenger abr28	facebook-ads, google-ads, outros, tiktok, youtube

Sinônimos que mapeamos do nosso lado (não precisa vocês mudarem): fb→facebook-ads, ig→facebook-ads, facebook→facebook-ads, google→google-ads, youtube-bio→youtube.

utm.medium — valores reconhecidos pelo modelo (Champion jan30, 7 categorias):

- Aberto
- Linguagem de programação
- Lookalike 1% Cadastrados - DEV 2.0 + Interesse Ciência da Computação
- Lookalike 2% Cadastrados - DEV 2.0 + Interesses
- Lookalike 2% Alunos + Interesse Linguagem de Programação
- dgen
- Outros

(Challenger abr28 é igual menos Lookalike 2% Alunos + Interesse Linguagem de Programação.)
Categorias raras são agrupadas em Outros por **limiar de frequência** (não por mapeamento de sinônimo).

Em produção real (UTMTracking, 30d) os valores de medium chegam com sufixo de criativo — exemplos reais:

- ABERTO
- ABERTO | AD0027-V1-REACT
- ABERTO | AD0152 JAN 2026
- ABERTO | AD0141-V2-REACT + HEADLINE
- ABERTO | AD0150-V3-REACT+HEADLINE

Esses sufixos NÃO estão na lista canônica; eles colapsam em Aberto (ou Outros) pela normalização nossa.
Mande exatamente como vem do anúncio — sufixo, espaços, e tudo. O nosso lado decide o agrupamento.

utm.term — valores reconhecidos pelo modelo: facebook, instagram, outros. Sinônimos: ig→instagram, fb→facebook; padrões --, {, ID numérico longo → outros.

utm.campaign, utm.content, utm.url — strings livres, não-católicas pro modelo. content é usado pelo Meta como nome do criativo (Ad name), então **importa**. url é a URL completa da página de captação.

A.4 Cuidados (do lado de vocês)

- hasComputer **é top-level**, NÃO dentro de survey.
- fbc **vazio**: enviar null, **nunca** "".
- **Macros do Facebook não-renderizadas** ({{adset.name}}, {{ad.name}}, {{site_source_name}}): **não enviar literais** — renderizar de verdade na configuração do anúncio, ou enviar "" se não houver valor.
- survey **não aceita campos extras**; pergunta extra por lançamento vai em bloco separado (ver §3), não dentro de survey.
- utm.source **pré-normalizado** (facebook/google em vez de facebook-ads/google-ads): **resolvido do nosso lado** com os sinônimos acima — não precisa mudar.

Anexo B — Como gerar o eventId

Regra única: **gerar uma vez, quando o lead é criado, e guardar junto do lead. Todo reenvio do mesmo lead usa o mesmo valor.** Nunca regenerar por tentativa/resubmissão (senão a dedup quebra — ver §1).

```
// no momento em que o lead é criado (1ª vez):
const eventId = crypto.randomUUID(); // ex.: "a1b2c3d4-e5f6-4789-abcd-ef0123456789"
// persistir eventId junto do registro do lead.
// em qualquer reenvio/retry: ler o eventId salvo, NÃO gerar outro.
```

`crypto.randomUUID()` existe em navegador moderno e em Node 16+. Qualquer UUID v4 serve; o que importa é ser **estável por lead**.

Anexo C — Implementação e envio

Arquitetura: o **browser** coleta os dados e manda pro **backend de vocês**; o backend monta o payload do Anexo A e **publica no nosso Pub/Sub** (server-side — credencial nunca no browser).

Onde publicar:

- Projeto: smart-ads-451319 · Tópico: lead-capture-ingest
- Conta de publicação: lead-capture-publisher@smart-ads-451319.iam.gserviceaccount.com (só publica, só nesse tópico)
- **Credencial:** vocês recebem **um arquivo** — a chave JSON dessa conta, ~2 KB — **uma vez**, por canal seguro (gerenciador de senha, link de segredo autodestrutivo ou arquivo cifrado; não e-mail/Slack em texto puro). Vocês **não acessam nada do nosso lado** — é só esse arquivo. No backend, apontem a variável `GOOGLE_APPLICATION_CREDENTIALS` para o caminho dele.

1) Browser — coletar e mandar pro backend de vocês:

```
function getCookie(n) {
  const m = document.cookie.match('(\\s*'+ n + '\\s*=\\s*([^;]+)');
  return m ? m.pop() : null;
}
const q = new URLSearchParams(location.search);
const payloadParcial = {
  email: form.email.value,
  firstName: form.firstName.value,
  lastName: form.lastName.value,
  phone: form.phone.value,
  hasComputer: form.temComputador.checked ? "SIM" : "NAO",
  fbp: getCookie("_fbp"),
  fbc: getCookie("_fbc"),
  userAgent: navigator.userAgent,
  survey: { /* as 10 respostas, strings exatas do parag. 3 */ },
  utm: {
    source: q.get("utm_source"),
    medium: q.get("utm_medium"),
    campaign: q.get("utm_campaign"),
    content: q.get("utm_content"),
    term: q.get("utm_term"),
    url: location.href
  }
};
// enviar payloadParcial para o backend de vocês (não direto pra nós).
```

2) Backend — montar e publicar (recomendado: biblioteca oficial; cuida de auth, base64 e retry):

```
// Node: npm i @google-cloud/pubsub
// env: GOOGLE_APPLICATION_CREDENTIALS=/caminho/da/chave.json
const { PubSub } = require("@google-cloud/pubsub");
const pubsub = new PubSub(); // projeto vem da própria chave
const evt = {
  ...payloadParcial,
  eventId, // o salvo (Anexo B), não um novo
  submittedAt: new Date().toISOString(),
  ip4: req.ip
};
await pubsub.topic("lead-capture-ingest").publishMessage({ json: evt });
// a lib já faz retry com backoff no publish.
```

Sem biblioteca — requisição HTTP crua (2 pegadinhas):

- Endereço: POST `https://pubsub.googleapis.com/v1/projects/smart-ads-451319/topics/lead-capture-ingest:publish`

- Authorization: Bearer ACCESS_TOKEN — **não** é a chave; é um token OAuth2 derivado dela (grant JWT-bearer da service account, escopo `https://www.googleapis.com/auth/pubsub`).
- Corpo: o JSON do Anexo A em UTF-8 **codificado em base64** dentro de data (não vai o JSON cru):

```
POST https://pubsub.googleapis.com/v1/projects/smart-ads-451319/topics/lead-capture-ingest:publish
Authorization: Bearer <token OAuth2 da service account>
Content-Type: application/json

{ "messages": [ { "data": "<base64(JSON do Anexo A)>" } ] }
```

Pelas 2 pegadinhas (minteio de token + base64), **use a biblioteca** salvo impedimento real.

Anexo D — Correções obrigatórias antes de ir ao ar

A produção atual faz três coisas erradas que o sistema novo **tem que** corrigir:

1. eventId **único**. Hoje há **dois ids diferentes para o mesmo lead** (survey_... numa tabela, lead_... noutra). No sistema novo: **um só**, gerado 1x e reusado em tudo (Anexo B).
2. fbc **vazio** → null. Produção grava "" quando não houve clique. Normalizar para null antes de enviar — não mandar string vazia.
3. **Macros de UTM renderizadas**. Caso real (19/05): o anúncio do Facebook não expandiu as variáveis e chegou o literal —

```
utm.source = "facebook-ads"          (ok, renderizado)
utm.medium = "{{adset.name}}"        (NÃO renderizado)
utm.content = "{{ad.name}}"          (NÃO renderizado)
utm.term   = "{{site_source_name}}"  (NÃO renderizado)
```

Garantir que as macros do Facebook sejam **de fato renderizadas** (parametrização correta da URL do anúncio). Sem valor → vazio (""); **nunca** o literal `{{. . .}}`. Não quebra do nosso lado (vira "outros"), mas cega a medição de desvio.

Base: *schemas e preenchimento medidos no Railway em 2026-05-19 (n=1.588 respondentes de pesquisa, 7d anteriores)*; *whitelist/allowlist de UTM em configs/clients/devclub.yaml*. Substitui o antigo `instrucoes_dev_frontend_capi.md` (*deprecado*).